

2024-2025

SAÉ 3

RECETTE 2

LUCCI & DALIGAUX

```
Microsoft Windows [version 10.0.22631.4662]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\geii>ping 192.168.1.60

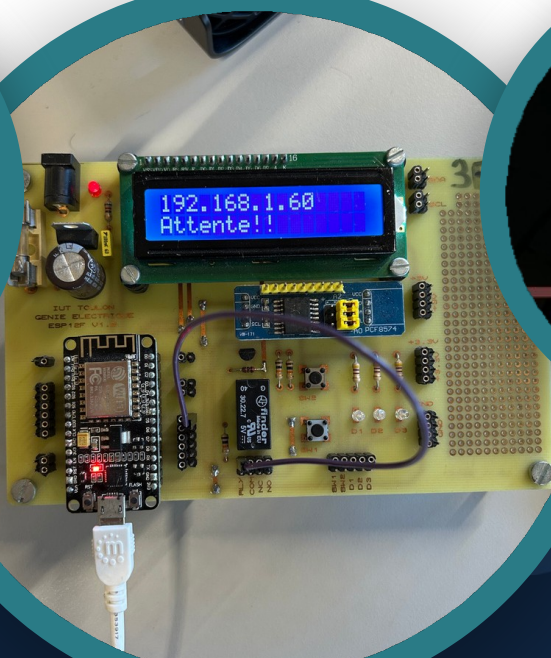
Envoi d'une requête 'Ping' : 192.168.1.60 avec 32 octets de données :
Réponse de 192.168.1.60 : octets=32 temps=7 ms TTL=255
Réponse de 192.168.1.60 : octets=32 temps=6 ms TTL=255
Réponse de 192.168.1.60 : octets=32 temps=70 ms TTL=255
Réponse de 192.168.1.60 : octets=32 temps=15 ms TTL=255

Statistiques Ping pour 192.168.1.60 :
    Paquets : envoyés = 4, reçus = 4, perdus = 0 (perte 0%),
    Durée approximative des boucles en millisecondes :
        Minimum = 6ms, Maximum = 70ms, Moyenne = 10ms

C:\Users\geii>ping lucci.local

Envoi d'une requête 'ping' sur lucci.local [192.168.1.60] avec 32
Réponse de 192.168.1.60 : octets=32 temps=7 ms TTL=255
Réponse de 192.168.1.60 : octets=32 temps=11 ms TTL=255
Réponse de 192.168.1.60 : octets=32 temps=0 ms TTL=255
Réponse de 192.168.1.60 : octets=32 temps=16 ms TTL=255

Statistiques Ping pour 192.168.1.60 :
    Paquets : envoyés = 4, reçus = 4, perdus = 0 (perte
    Durée approximative des boucles en millisecondes :
        Minimum = 7ms, Maximum = 16ms, Moyenne = 11ms
```



est Unitaire PWM

Commande Relai

Mettre à 0 Allumer au max

Statut du Relai : OFF

PWM

Valeur du PWM : 0% Actualiser PWM

TU 1 – Programme Labview avec un serveur TCP

L'objectif de ce test est de communiquer depuis le programme labview sur un serveur TCP généré avec netcat. Les critères de réussites de ce test Unitaire sont de générer un serveur TCP et de pouvoir envoyer une valeur de 0 à 100 depuis le programme labview sur le serveur TCP.

Tout d'abord, il faut récupérer l'ip de l'ordinateur sur lequel le port va être ouvert. Avec la commande ifconfig, donc ici l'ip est 192.168.1.72. Avec la commande nc -lk 4500, le port est ouvert sur le 4500. L'ip ainsi que le port est rentré dans le programme labview permettant de rentrer dans la communication TCP

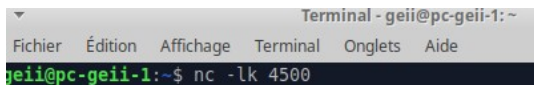


Figure 1 : Ouverture du port



Figure 2 : Labview IP et Port

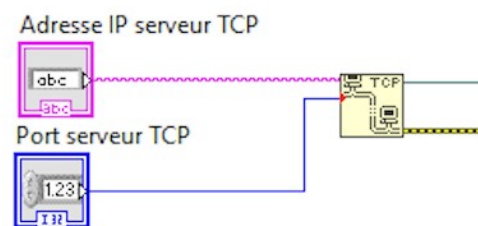


Figure 3 : Labview connection IP et Port sur le TCP

Le programme labview est réalisé par le block (TCP) ci-dessus permettant de rentrer dans la communication. La fermeture est effectuée par le bloc suivant.



Figure 3 : Labview fermeture communication TCP

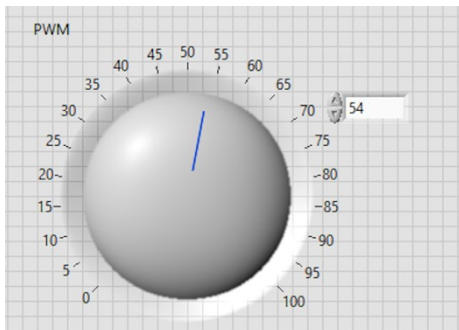


Figure 5 : Labview molette et entrée de valeur

Le but va être d'envoyer la valeur de cette roulette ou de l'entrée de valeur (permettant d'être plus précis) sur le serveur TCP. Pour cela un bloc écriture TCP est rajouté entre l'ouverture et la fermeture permettant ainsi d'entrer la valeur souhaitée

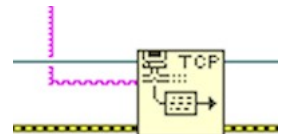


Figure 4 : Labview bloc écriture TCP

Pour cela il faut ajouter 2 blocs le premier permettant la conversion de donnée prenant une valeur numérique et la transformant en une chaîne caractères (type String). Le deuxième bloc permet de concaténer la chaîne de caractère pour la préparer à l'envoi.

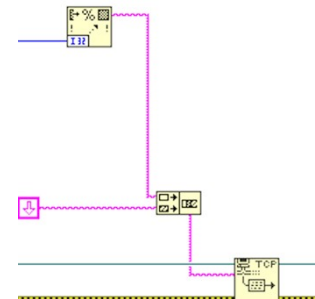


Figure 6 : Labview conversion de donnée et concaténer la chaîne de caractère

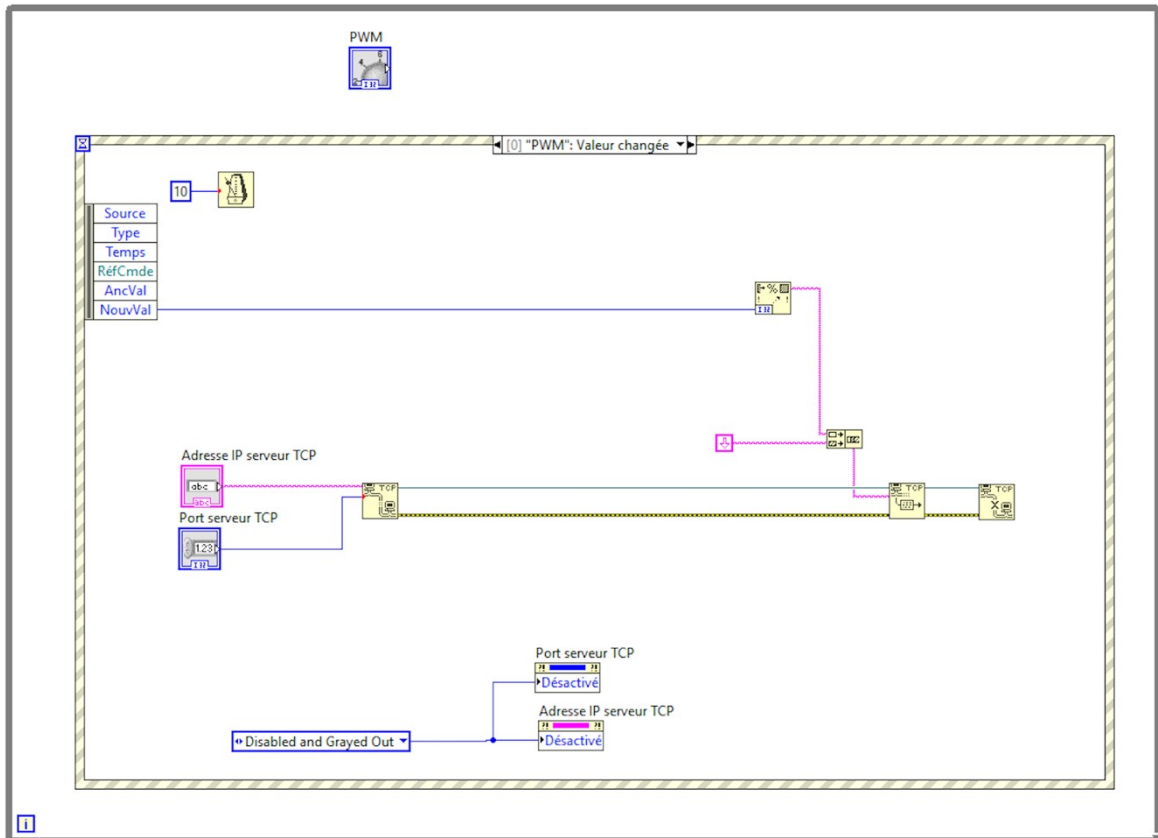


Figure 7 : Labview Structure événement

La structure événement permet l'activation de son contenu à l'unique condition que la variable "PWM" ai changé de valeur, cette valeur est donc reprise ensuite dans "NouvVal" puis est envoyé dans le bloc d'écriture TCP. Cette structure empêche la répétition d'envoi d'information identique.

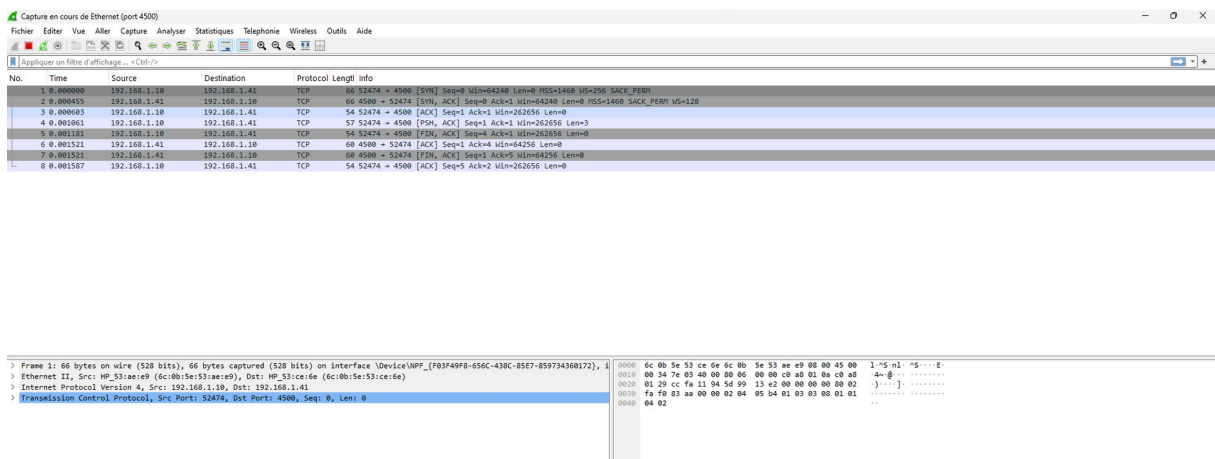


Figure 8 : Visualisation communication wireshark

Cela est bien observable sur la figure ci-dessus, la communication ne se répète donc pas à l'infini.

```

Terminal - geii@pc-geii-1: ~
Fichier Édition Affichage Terminal Onglets Aide
geii@pc-geii-1:~$ nc -lk 4500
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

En changeant la valeur du PWM, elle est bien affichée sur le serveur TCP au port 4500. En affichant seulement la valeur lorsqu'elle est changée. Le test Unitaire est donc validé.

Figure 9 : Visualisation valeur PWM sur netcat

TU 2 – Commande telnet sur un serveur TCP sur l'ESP

Via la commande telnet modifier la valeur du PWM sur le serveur TCP généré par l'ESP
Les critères de réussites sont de générer un serveur TCP sur l'ESP et de pouvoir modifier la valeur du PWM de la carte via une commande sur un terminal (commande telnet).

```
if (!client.connected()) {  
  
    client = servertcp.available();  
    if (client) {  
        cmd = client.readStringUntil('\n');  
        client.stop();  
        pwmval = cmd.toInt();  
        if (pwmval < 0) { pwmval = 0; }  
        if (pwmval > 100) { pwmval = 100; }  
        pwmcval = pwmval * 2.55;  
        analogWrite(PwmPin, pwmcval);  
    }  
}
```

Figure 10 : Arduino vérification client et lecture de donnée

Le premier if va permettre de vérifier si le client est correctement attribué avant de l'utiliser. Donc si le client est valide le deuxième est possible, la lecture des données va se faire avec le `client.readStringUntil('\n')`, la donnée sera lue à partir du moment où la touche d'entrée est activée grâce au `\n`. Par la suite les valeurs sont lues grâce au `cmd.ToInt()` qui est mis dans la variable `pwmval`. S'il est inférieur à 0, il est mis à 0 pour éviter d'avoir du négatif qui ne pourra pas être utilisé par la suite et s'il est supérieur à 100 il sera mis à 100 car la valeur souhaitée est un pourcentage. Le calcul du PWM est effectué à la suite.

```
lcd.setCursor(0, 1);  
    lcd.print("PWM : ");  
    lcd.print(" ");  
    lcd.setCursor(6, 1);  
    lcd.print(pwmval);  
    lcd.print("%");  
    lcd.setCursor(0,0);  
    lcd.print(" ");  
    lcd.setCursor(0,0);  
    lcd.print(WiFi.localIP()); lcd.print(" ");
```

Figure 11 : Arduino affichage sur l'écran LCD

Par la suite la valeur souhaitée est affichée sur l'écran LCD.

Après plusieurs tests, la valeur souhaitée s'affiche bien sur l'écran LCD, les tests permettant de maintenir la valeur du PWM de 0 à 100% marche correctement. Le test unitaire est donc validé.

```

Terminal - geii@pc-geii-1: ~
Fichier  Édition  Affichage  Terminal  Onglets  Aide
Trying 192.168.1.68...
Connected to 192.168.1.68.
Escape character is '^]'.
40
Connection closed by foreign host.
geii@pc-geii-1:~$ telnet 192.168.1.68 4500
Trying 192.168.1.68...
Connected to 192.168.1.68.
Escape character is '^]'.
dza
Connection closed by foreign host.
geii@pc-geii-1:~$ telnet 192.168.1.68 4500
Trying 192.168.1.68...
Connected to 192.168.1.68.
Escape character is '^]'.
516651564
Connection closed by foreign host.
geii@pc-geii-1:~$ telnet 192.168.1.68 4500
Trying 192.168.1.68...
Connected to 192.168.1.68.
Escape character is '^]'.
90
Connection closed by foreign host.
geii@pc-geii-1:~$

```

Figure 12 : Telnet lecture valeur PWM de 0 à 100 %

TI 1 – Programme LabVIEW modifiant PWM sur serveur TCP sur ESP

Le programme LabVIEW doit pouvoir accéder à un serveur TCP généré par l'ESP et via le LabVIEW la modification du PWM doit être possible. Les critères de réussite sont : La possibilité de se connecter au serveur TCP de l'ESP via LabVIEW ainsi que de faire varier le PWM de 0 à 100% sur la visualisation LabVIEW.

Le programme LabView ainsi que le programme arduino reste les mêmes que pour le TU1 pour labview et TU2 pour arduino. Mais les critères de réussites sont différents.

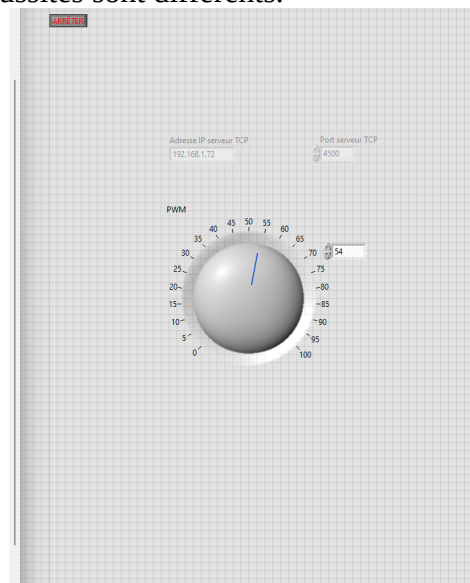
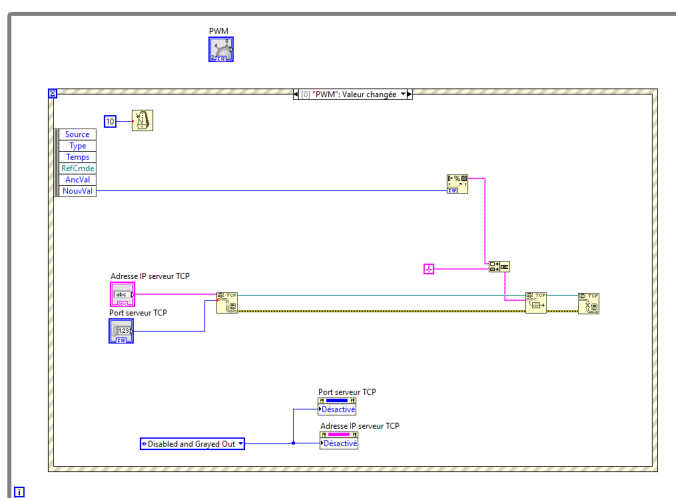


Figure 13 : Labview programme

TI 2 – Intégration de serveur TCP et WEB sur ESP permettant la variation du PWM

Le PWM de la carte doit pouvoir être modifié par interface WEB ou via programme LabVIEW ou par commande telnet sans interférer entre eux. Les critères de réussite sont la génération sur l'ESP d'un serveur WEB et un serveur TCP ainsi que la variation de 0 à 100% du PWM de la carte grâce à la page web, le programme LabVIEW et les commandes telnet.

La création de la page web en html ainsi que la fonction handleRoot comme effectué dans le TI1 de la recette précédente est rajouté dans le programme arduino

```
String getPage() {
    String page = "<html>";
    page += "<head>";
    page += "<title>Test Integration PWM</title>";
    page += "<style>";
    ...
    // Slider container
    page += "<div class='slider-container'>";
    page += "<h3> PWM </h3>";
    page += "<form action='/' method='POST'>";
    page += "<input type='range' name='PWM' min='0' max='100' value='" + String(pwmval) +
    "'oninput='this.nextElementSibling.value = this.value'>";
    page += "<output>" + String(pwmval) + "</output>";
    page += "<div class='slider-label'>Valeur du PWM : <strong>" + String(pwmval) +
    "%</strong></div>";
    ...
    return page;
}
```

Figure 14 : Page Web en HTML

```
void handleRoot(){
    String postValue;
    if ( server.hasArg("RLY")) {
        postValue = server.arg("RLY");
        if ( postValue == "1" ) {
            stateRelay = HIGH;
            pwmval = 100;
            pwmcval = 255;
            digitalWrite(cmdeRelay, stateRelay);
            lcd.setCursor(0,0);
            lcd.print("Relay ON      ");
            server.send ( 200, "text/html", getPage() );
        } else
            if ( postValue == "0" ) {
                stateRelay = LOW;
                pwmval = pwmcval = 0;
                digitalWrite(cmdeRelay, stateRelay);
                lcd.setCursor(0,0);
                lcd.print("Relay OFF      ");
                server.send ( 200, "text/html", getPage() );
            }
    }
}
```

```

    }

}

if (server.hasArg("PWM")) {
    pwmval = server.arg("PWM").toInt();
    if(pwmval >= 1){stateRelay = HIGH;digitalWrite(cmdRelay,
stateRelay);lcd.setCursor(0,0);lcd.print("Relay ON ");}
    pwmcalc = pwmval*2.55;
    analogWrite(PwmPin,pwmcalc);
    lcd.setCursor(0,1);
    lcd.print("PWM : ");
    lcd.setCursor(8,1);
    lcd.print("  ");
    lcd.setCursor(6,1);
    lcd.print(pwmval);
    lcd.print("%");
    server.send ( 200, "text/html", getPage() );
}

server.send ( 200, "text/html", getPage() );
}

//////////////////// Error 404 //////////////////////
void handleNotFound() {
    String message = "<html> <body> <H1> <center> <BR> <BR> Error 404, File Not Found on
this server</center> </H1> </body> </html>";
    server.send(404, "text/html", message);
}

```

Figure 15 : Calcul PWM page WEB

La création ainsi que la génération de la page sont donc rajoutées dans le programme avec la connexion au serveur TCP expliqué précédemment.

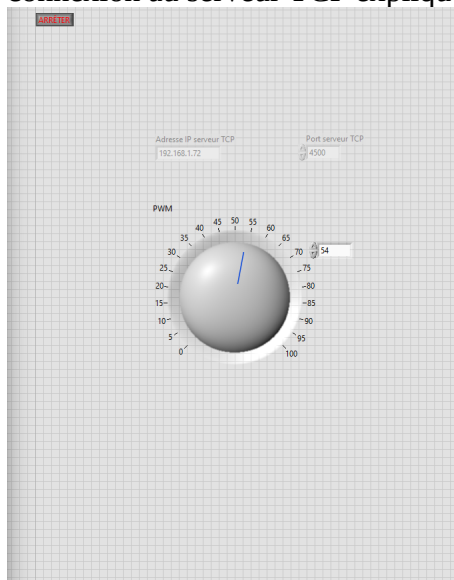


Figure 16 : Labview contrôle du signal

```

Terminal - geii@pc-geii-1: ~
Fichier  Édition  Affichage  Terminal  Onglets  Aide

Trying 192.168.1.68...
Connected to 192.168.1.68.
Escape character is '^]'.
40
Connection closed by foreign host.
geii@pc-geii-1:~$ telnet 192.168.1.68 4500
Trying 192.168.1.68...
Connected to 192.168.1.68.
Escape character is '^]'.
dza
Connection closed by foreign host.
geii@pc-geii-1:~$ telnet 192.168.1.68 4500
Trying 192.168.1.68...
Connected to 192.168.1.68.
Escape character is '^]'.
516651564
Connection closed by foreign host.
geii@pc-geii-1:~$ telnet 192.168.1.68 4500
Trying 192.168.1.68...
Connected to 192.168.1.68.
Escape character is '^]'.
90
Connection closed by foreign host.
geii@pc-geii-1:~$

```

Figure 15 : Telnet contrôle du signal

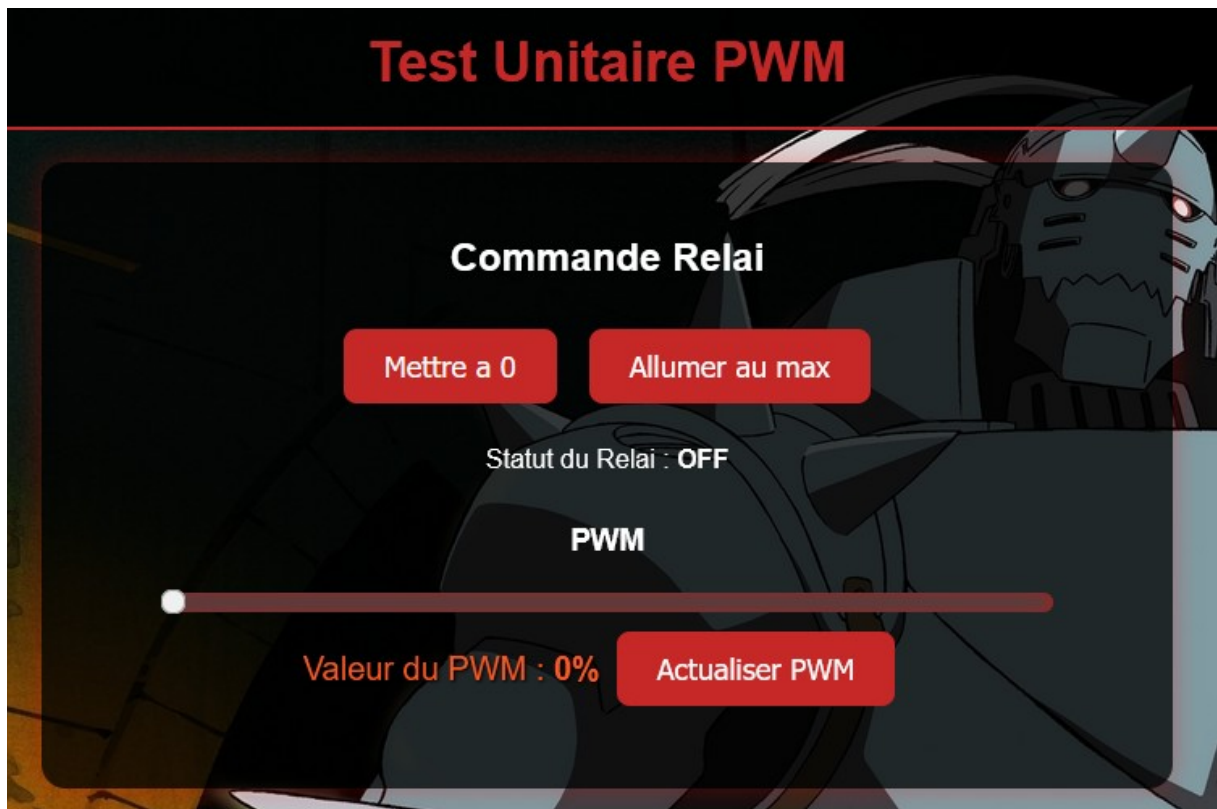


Figure 15 : Page Web contrôle du signal

Ce dernier test permet ainsi de gérer le signal PWM sur les 3 interfaces suivantes en même temps sans la nécessité de relancer le programme.

Le test unitaire est donc validé, la possibilité de gérer le signal PWM par les 3 interfaces ci-dessus avec la visualisation du signal sur oscilloscope permet de validé ce test.